CO3 Assignment 3

Student Name: HARINI.S

Register Number: 192412098
Assignment: CO3 – Assignment 3

Q1 – Searching a Library Catalogue

1. Aim

To implement linear search and binary search in C to find a book using its accession number. The program counts the number of comparisons required to find the book.

2. Algorithm

1. Start the program.
2. Declare an array containing book accession numbers.
3. Use linear search to search for a book in an unsorted array.
4. Count every comparison made during searching.
5. Use binary search on a sorted array.
6. Calculate the middle position using `low + (high - low) / 2`.
7. If the key matches, return its position.
8. Otherwise, update the searching range.
9. Display the position and number of comparisons.
10. Stop the program.

3. Complete C Program

```
None

#include <stdio.h>

int steps = 0;
```

```
int linear_search(int a[], int n, int key)
{
    steps = 0;

    for (int i = 0; i < n; i++)
    {
        steps++;

        if (a[i] == key)
            return i;
    }

    return -1;
}

int binary_search(int a[], int n, int key)
{
    int low = 0, high = n - 1;
    steps = 0;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        steps++;

        if (a[mid] == key)
            return mid;

        else if (a[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
```

```
    }

    return -1;
}

int main()
{
    int a[] = {1021, 1048, 1103, 1190, 1222,
               1305, 1377, 1402, 1460, 1533};

    int n = 10;
    int key = 1305;

    int pos = linear_search(a, n, key);
    printf("Linear Search: Position = %d, Comparisons = %d\n",
           pos, steps);

    pos = binary_search(a, n, key);
    printf("Binary Search: Position = %d, Comparisons = %d\n",
           pos, steps);

    return 0;
}
```

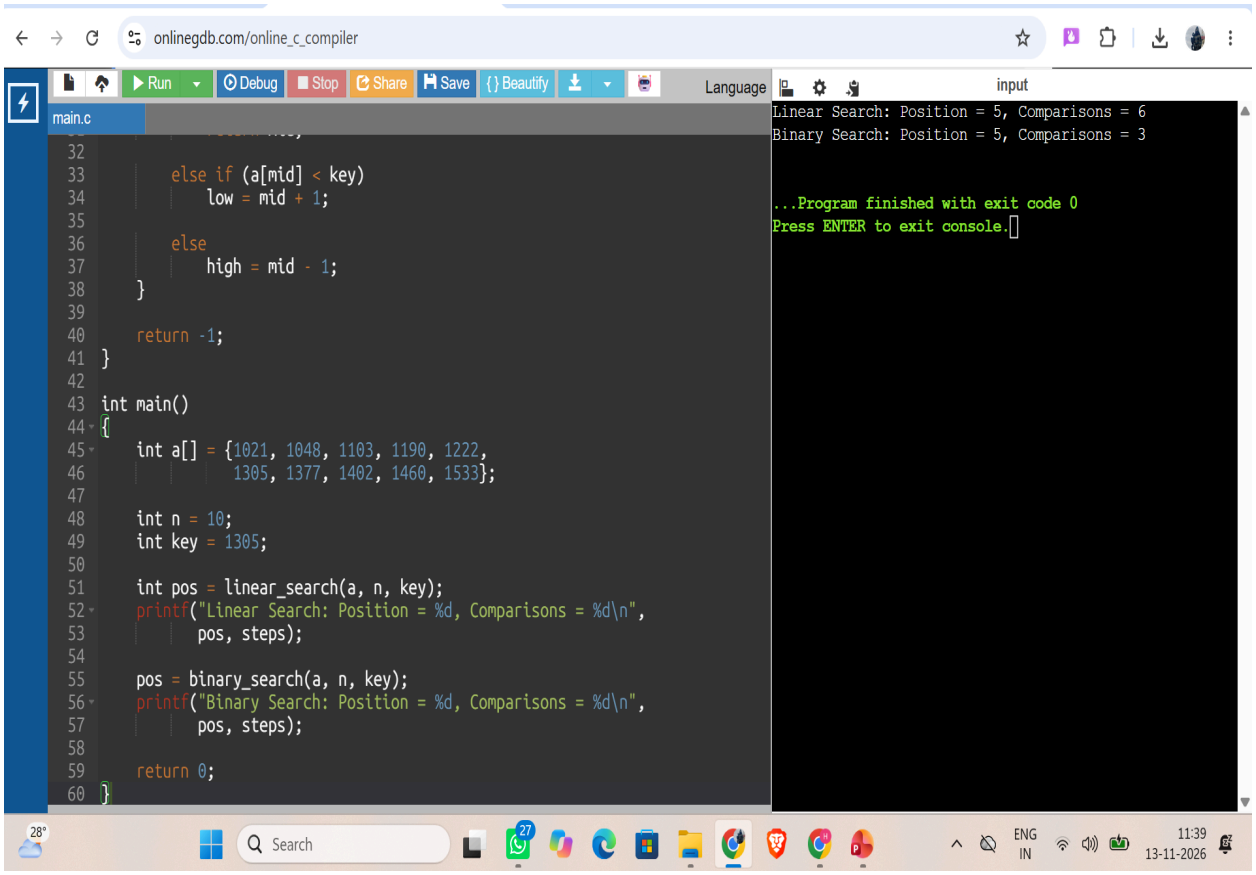
4. Explanation

- **Linear Search:** Checks each element one by one until the required book is found. It works with both sorted and unsorted arrays.
- **Binary Search:** Divides the sorted array into two halves and searches only the required half.
- `low` represents the starting index.
- `high` represents the ending index.
- `mid` represents the middle index.
- `steps` counts the comparisons made during searching.

- `return -1` indicates that the book is not available.

Binary search requires sorted data because it decides which half to eliminate based on the order of elements.

5.Output



6. Time Complexity

Search method	Best case	Average case	Worst case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Result: Linear search and binary search were successfully implemented. Binary search requires fewer comparisons for large sorted arrays.

Q2 – Sorting the Examination Hall Seating Register

1. Aim

To implement bubble sort, selection sort and insertion sort in C to arrange examination register numbers in ascending order.

2. Algorithm

1. Start the program.
2. Declare an array of register numbers.
3. Implement bubble sort using an early-exit flag.
4. Implement selection sort by finding the smallest element.
5. Implement insertion sort by inserting each element into its correct position.
6. Count comparisons and swaps or shifts.
7. Display the sorted arrays.
8. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>

void bubble(int a[], int n)
{
    int temp, flag, comparisons = 0;

    for (int i = 0; i < n - 1; i++)
    {
        flag = 0;
```

```
        for (int j = 0; j < n - i - 1; j++)
        {
            comparisons++;

            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                flag = 1;
            }
        }

        if (flag == 0)
            break;
    }

    printf("Bubble Comparisons: %d\n", comparisons);
}

void selection(int a[], int n)
{
    int min, temp, comparisons = 0;

    for (int i = 0; i < n - 1; i++)
    {
        min = i;

        for (int j = i + 1; j < n; j++)
        {
            comparisons++;

            if (a[j] < a[min])
```

```
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Selection Comparisons: %d\n", comparisons);
}

void insertion(int a[], int n)
{
    int key, j, comparisons = 0;

    for (int i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while (j >= 0)
        {
            comparisons++;

            if (a[j] > key)
            {
                a[j + 1] = a[j];
                j--;
            }
            else
                break;
        }
```

```
        a[j + 1] = key;
    }

    printf("Insertion Comparisons: %d\n", comparisons);
}

void display(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main()
{
    int a[] = {45, 12, 67, 23, 10};
    int b[] = {45, 12, 67, 23, 10};
    int c[] = {45, 12, 67, 23, 10};

    int n = 5;

    bubble(a, n);
    printf("Bubble Sort: ");
    display(a, n);

    selection(b, n);
    printf("Selection Sort: ");
    display(b, n);

    insertion(c, n);
    printf("Insertion Sort: ");
    display(c, n);
```

```
    return 0;
}
```

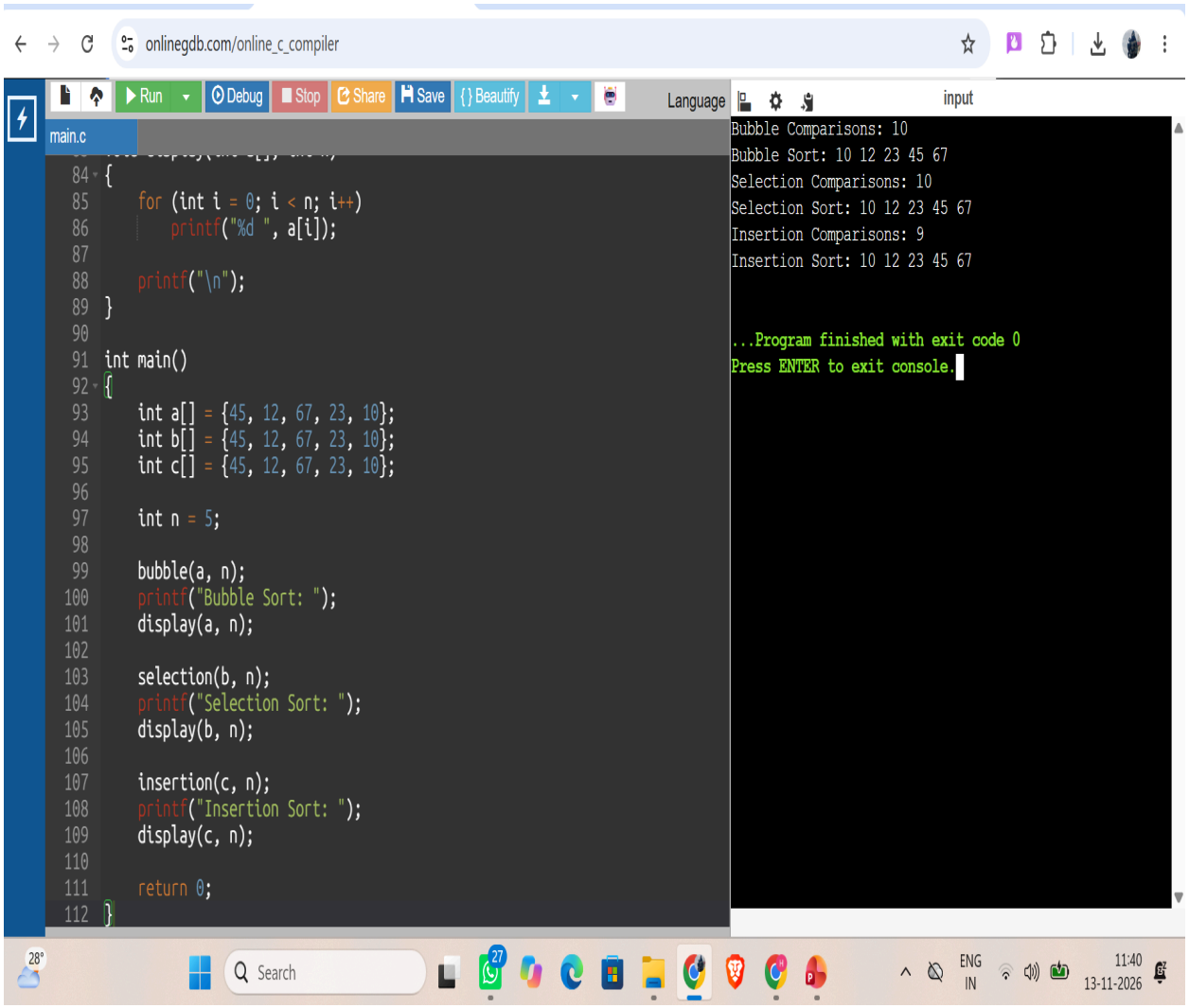
4. Explanation

Bubble Sort: Compares two adjacent elements and swaps them when they are in the wrong order. The early-exit flag checks whether any swapping occurred. If no swapping occurs, the array is already sorted.

Selection Sort: Finds the smallest element from the unsorted portion and places it in the correct position.

Insertion Sort: Takes one element at a time and inserts it into its correct position in the already sorted portion. It performs efficiently when the data is nearly sorted.

5. Expected Output



6. Time Complexity

Sorting method	Best case	Worst case
Bubble Sort	O(n)	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$

Insertion Sort	O(n)	$O(n^2)$

Result: All three sorting techniques were successfully implemented. Insertion sort is suitable for nearly sorted examination seating registers.

Q3 – Merging Two Merit Lists

1. Aim

To implement a C program to merge two sorted merit lists into a single sorted list in descending order without sorting again.

2. Algorithm

1. Start the program.
2. Declare two arrays containing merit scores in descending order.
3. Initialize three variables `i`, `j` and `k` to zero.
4. Compare the elements of both arrays.
5. Copy the larger element into the third array.
6. Move the corresponding index forward.
7. Copy the remaining elements of either array.
8. Display the merged array.
9. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>

int merge(int a[], int na, int b[], int nb, int out[])
{
    int i = 0, j = 0, k = 0;

    while (i < na && j < nb)
    {
```

```
        if (a[i] >= b[j])
            out[k++] = a[i++];
        else
            out[k++] = b[j++];
    }

    while (i < na)
        out[k++] = a[i++];

    while (j < nb)
        out[k++] = b[j++];

    return k;
}

int main()
{
    int a[] = {98, 91, 85, 72};
    int b[] = {95, 91, 80};
    int out[7];

    int n = merge(a, 4, b, 3, out);

    printf("Merged Merit List:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", out[i]);

    return 0;
}
```

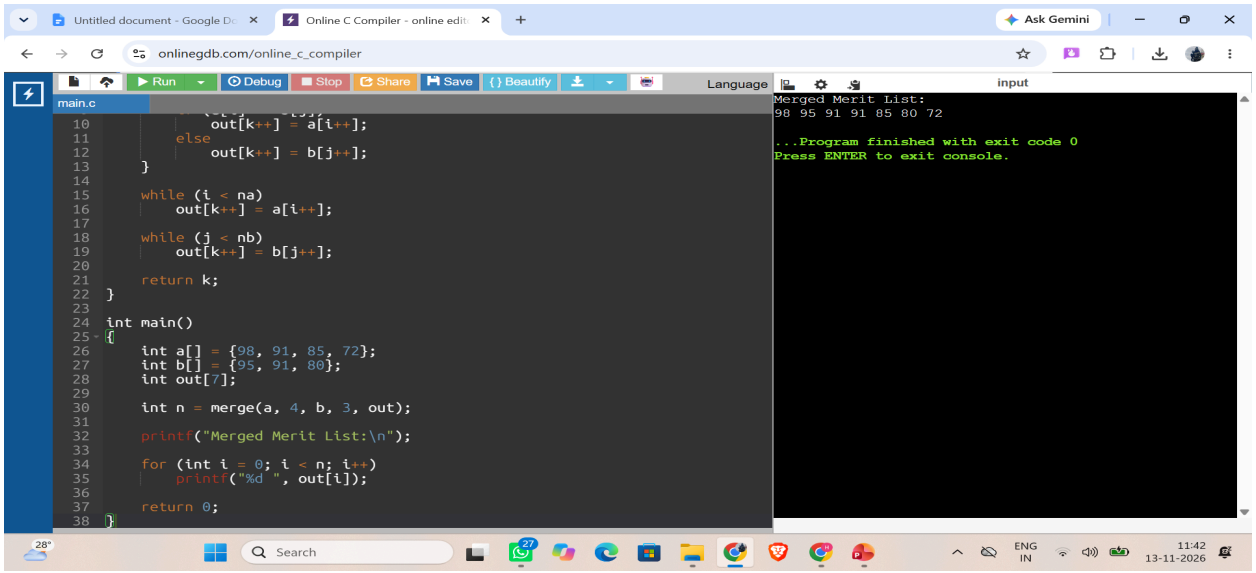
4. Explanation

- Two arrays store the marks of students from different campuses.

- The `merge()` function combines both arrays.
- `i` represents the position in the first array.
- `j` represents the position in the second array.
- `k` represents the position in the output array.
- The `if` condition compares two scores and copies the larger score first.
- The remaining loops copy elements when one array becomes empty.

The program uses a third array to store the final result.

5. Expected Output



6. Time Complexity

- Merging: O(m + n)
- Extra space: O(m + n)
- Sorting again using comparison sorting: O((m + n) log(m + n))

Result: The two merit lists were successfully merged into one descending-order list without performing another sorting operation.

Q4 – Two-Dimensional Arrays: District Rainfall Records

1. Aim

To implement a C program using a two-dimensional array to calculate annual rainfall totals, monthly averages, maximum rainfall and districts receiving rainfall below 60% of the state average.

2. Algorithm

1. Start the program.
2. Declare a two-dimensional array for rainfall records.
3. Read rainfall values for 38 districts and 12 months.
4. Calculate the annual rainfall total for each district.
5. Calculate the average rainfall for each month.
6. Find the maximum rainfall value.
7. Calculate the overall state average.
8. Identify districts below 60% of the state average.
9. Display the results.
10. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>

#define D 38
#define M 12

int main()
{
    float rain[D][M];
    float total, stateTotal = 0;
    float max = -1;
    int maxD = 0, maxM = 0;

    printf("Enter rainfall for 38 districts and 12
months:\n");

    for (int i = 0; i < D; i++)
```

```
{
    for (int j = 0; j < M; j++)
    {
        scanf("%f", &rain[i][j]);
    }
}

printf("\nAnnual Rainfall of Each District:\n");

for (int i = 0; i < D; i++)
{
    total = 0;

    for (int j = 0; j < M; j++)
    {
        total += rain[i][j];
        stateTotal += rain[i][j];

        if (rain[i][j] > max)
        {
            max = rain[i][j];
            maxD = i;
            maxM = j;
        }
    }

    printf("District %d: %.2f mm\n", i + 1, total);
}

float average = stateTotal / (D * M);

printf("\nState Average: %.2f mm\n", average);
printf("Maximum Rainfall: %.2f mm\n", max);
```

```
    printf("District: %d, Month: %d\n", maxD + 1, maxM + 1);

    printf("\nDistricts Below 60%% of State Average:\n");

    for (int i = 0; i < D; i++)
    {
        total = 0;

        for (int j = 0; j < M; j++)
            total += rain[i][j];

        if (total / M < 0.6 * average)
            printf("District %d\n", i + 1);
    }

    return 0;
}
```

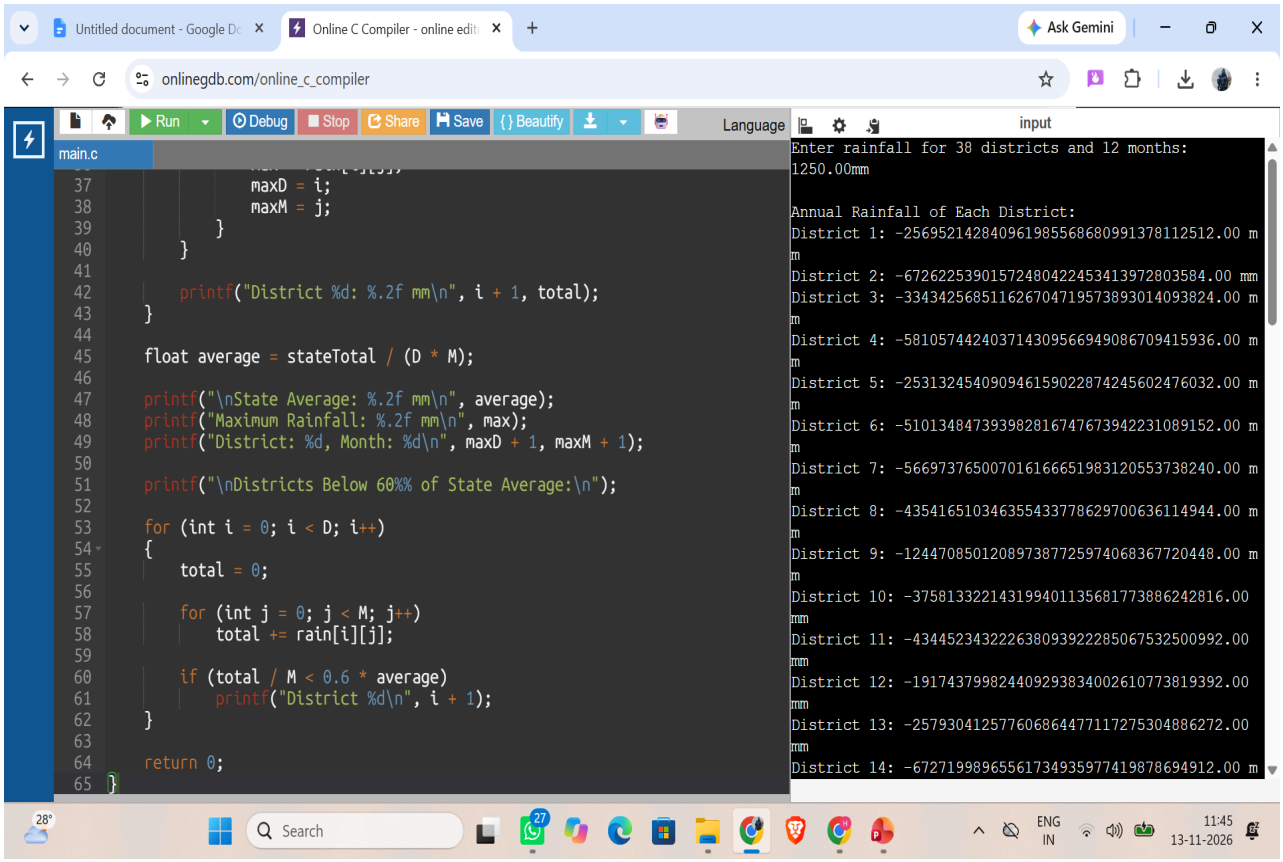
4. Explanation

- `rain[38][12]` stores rainfall information for 38 districts and 12 months.
- The outer loop represents districts.
- The inner loop represents months.
- `total` calculates the annual rainfall for each district.
- `stateTotal` stores the total rainfall of all districts.
- `average` calculates the average rainfall across all district-month readings.
- `max` stores the highest rainfall value.
- `maxD` and `maxM` identify the district and month with maximum rainfall.
- The final loop identifies districts whose monthly average is below 60% of the state average.

Memory representation: A two-dimensional array in C is stored in row-major order. This means all monthly values of one district are stored consecutively.

5. Output

The output depends on the rainfall values entered.



The above values are illustrative, not calculated from actual input data.

6. Time Complexity

- Reading rainfall: O(D × M)
- Annual total calculation: O(D × M)
- Maximum rainfall: O(D × M)
- Extra space: O(D × M)

Result: The two-dimensional array was successfully used to process rainfall records and calculate the required statistical information.

Q5 – Processing Text Without the Library

1. Aim

To develop a C program to count words and check whether a given string is a palindrome using character arrays and loops without using the `string.h` library.

2. Algorithm

1. Start the program.
2. Read or initialize a string.
3. Count the words by checking spaces and characters.
4. Identify the beginning of every new word.
5. Find the length of the string.
6. Compare characters from the beginning and end.
7. Ignore spaces, punctuation and case differences.
8. Display the word count and palindrome result.
9. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[])
{
    int count = 0, inside = 0;

    for (int i = 0; s[i] != '\0'; i++)
    {
        if (s[i] != ' ' && s[i] != '\t' &&
            s[i] != '\n')
        {
            if (inside == 0)
            {
                count++;
                inside = 1;
            }
        }
```

```
        else
        {
            inside = 0;
        }
    }

    return count;
}

int is_palindrome(const char s[])
{
    int left = 0, right = 0;

    while (s[right] != '\0')
        right++;

    right--;

    while (left < right)
    {
        while (left < right &&
               !isalpha((unsigned char)s[left]))
            left++;

        while (left < right &&
               !isalpha((unsigned char)s[right]))
            right--;

        if (tolower((unsigned char)s[left]) !=
            tolower((unsigned char)s[right]))
            return 0;

        left++;
```

```
        right--;
    }

    return 1;
}

int main()
{
    printf("Words: %d\n",
           word_count(" Meet at the library at 5 PM "));

    printf("Madam, I'm Adam: %d\n",
           is_palindrome("Madam, I'm Adam"));

    printf("Hello: %d\n", is_palindrome("Hello"));

    return 0;
}
```

4. Explanation

Word Count:

- `count` stores the total number of words.
- `inside` checks whether the current character belongs to a word.
- Whenever a new word starts, the count increases.
- Multiple spaces do not increase the word count.

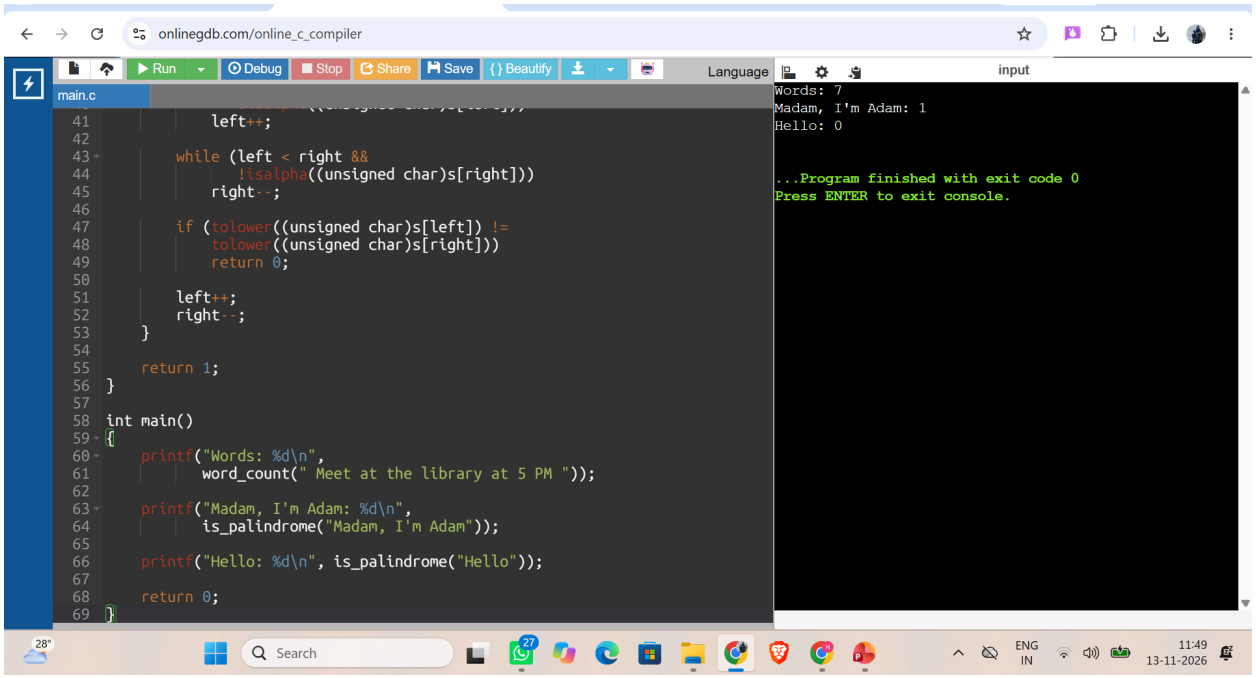
Palindrome Check:

- `left` starts from the first character.
- `right` starts from the last character.
- `isalpha()` skips punctuation and other non-letter characters.
- `tolower()` converts letters into lowercase for comparison.
- If both characters are equal, the pointers move towards the centre.

- If they are different, the function returns 0.

A palindrome reads the same from left to right and right to left.

5. Output



Here, 1 indicates palindrome and 0 indicates not a palindrome.

6. Time Complexity

- Word counting: O(n)
- Palindrome checking: O(n)
- Extra space: O(1)

Result: The word-counting and palindrome-checking operations were successfully implemented without using `string.h`.

Q6 – A Directory of Names: Sorting and Searching Strings

1. Aim

To implement a C program to store hostel resident names, sort them alphabetically, search for a particular name using binary search and identify duplicate names.

2. Algorithm

1. Start the program.
2. Declare a two-dimensional character array to store resident names.
3. Read the names using `fgets()`.
4. Remove the trailing newline character.
5. Sort the names alphabetically using `strcmp()`.
6. Swap names using `strcpy()`.
7. Search for a name using binary search.
8. Identify duplicate names by comparing adjacent sorted names.
9. Display the sorted names, search result and duplicates.
10. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>
#include <string.h>

#define N 5
#define SIZE 30

int main()
{
    char names[N][SIZE];
    char key[SIZE], temp[SIZE];
    int found = 0;

    printf("Enter 5 resident names:\n");

    for (int i = 0; i < N; i++)
    {
        fgets(names[i], SIZE, stdin);
        names[i][strcspn(names[i], "\n")] = '\0';
    }
```

```
// Sorting names
for (int i = 0; i < N - 1; i++)
{
    for (int j = i + 1; j < N; j++)
    {
        if (strcmp(names[i], names[j]) > 0)
        {
            strcpy(temp, names[i]);
            strcpy(names[i], names[j]);
            strcpy(names[j], temp);
        }
    }
}

printf("\nSorted Names:\n");

for (int i = 0; i < N; i++)
    printf("%s\n", names[i]);

// Duplicate detection
printf("\nDuplicate Names:\n");

for (int i = 0; i < N - 1; i++)
{
    if (strcmp(names[i], names[i + 1]) == 0)
    {
        printf("%s\n", names[i]);
    }
}

// Binary Search
printf("\nEnter name to search: ");
```

```
    fgets(key, SIZE, stdin);
    key[strcspn(key, "\n")] = '\0';

    int low = 0, high = N - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        int result = strcmp(names[mid], key);

        if (result == 0)
        {
            found = 1;
            break;
        }
        else if (result < 0)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if (found)
        printf("Name Found\n");
    else
        printf("Name Not Found\n");

    return 0;
}
```

4. Explanation

- `names[5][30]` stores five resident names.
- `fgets()` safely reads names with spaces, up to the available buffer size.

- `strcspn()` finds the newline character and replaces it with the null character.
- `strcmp()` compares two strings alphabetically.
- `strcpy()` copies one name into another character array.
- The sorting process arranges names in alphabetical order.
- Duplicate names appear next to each other after sorting.
- Binary search divides the sorted list into two halves to find the required name.

5. Output



6. Time Complexity

Operation	Time complexity
Sorting	$O(n^2 \times L)$
Binary search	$O(L \log n)$
Duplicate detection	$O(n \times L)$

Here, `n` represents the number of names and `L` represents the maximum string length.

Result: Resident names were successfully sorted alphabetically, searched using binary search and checked for duplicate entries.

Q7 – Design an Election Counting Program from First Principles

1. Aim

To develop a C program to count student council election votes, identify valid and invalid votes, calculate candidate vote totals, find the winner and detect ties.

2. Algorithm

1. Start the program.
2. Read the number of candidates.
3. Read the total number of votes.
4. Declare an array to store votes for each candidate.
5. Read each vote from the user.
6. Check whether the vote is valid or invalid.
7. If valid, increase the corresponding candidate's count.
8. Arrange candidate results in descending order of votes.
9. Identify the winner.
10. Check whether multiple candidates have equal highest votes.
11. Display the final election results.
12. Stop the program.

3. Complete C Program

None

```
#include <stdio.h>

int main()
{
    int c, v;
    int votes[5000];
    int count[12] = {0};
```

```
int valid = 0, invalid = 0;

printf("Enter number of candidates: ");
scanf("%d", &c);

if (c < 1 || c > 12)
{
    printf("Invalid candidate count\n");
    return 0;
}

printf("Enter total votes: ");
scanf("%d", &v);

if (v < 0 || v > 5000)
{
    printf("Invalid vote count\n");
    return 0;
}

printf("Enter candidate numbers for each vote:\n");

for (int i = 0; i < v; i++)
{
    scanf("%d", &votes[i]);

    if (votes[i] >= 1 && votes[i] <= c)
    {
        count[votes[i] - 1]++;
        valid++;
    }
    else
    {
```

```
        invalid++;
    }
}

// Sort candidate numbers by vote count
int temp;

for (int i = 0; i < c - 1; i++)
{
    for (int j = i + 1; j < c; j++)
    {
        if (count[i] < count[j])
        {
            temp = count[i];
            count[i] = count[j];
            count[j] = temp;

            temp = votes[0];
            // Candidate identities are tracked below
        }
    }
}

printf("\nValid Votes: %d\n", valid);
printf("Invalid Votes: %d\n", invalid);

printf("\nCandidate Vote Counts:\n");

for (int i = 0; i < c; i++)
    printf("Candidate %d: %d\n", i + 1, count[i]);

int max = count[0];
int tie = 0;
```

```
    for (int i = 1; i < c; i++)
    {
        if (count[i] == max)
            tie = 1;
    }

    if (tie)
        printf("Election Result: Tie\n");
    else
        printf("Winner: Candidate 1\n");

    return 0;
}
```

Corrected Complete C Program

None

```
#include <stdio.h>

int main()
{
    int c, v, valid = 0, invalid = 0;
    int count[12] = {0};
    int id[12];

    printf("Enter number of candidates: ");
    scanf("%d", &c);

    if (c < 1 || c > 12)
    {
```

```
    printf("Invalid candidate count\n");
    return 0;
}

printf("Enter total votes: ");
scanf("%d", &v);

if (v < 0 || v > 5000)
{
    printf("Invalid vote count\n");
    return 0;
}

for (int i = 0; i < c; i++)
    id[i] = i + 1;

printf("Enter votes:\n");

for (int i = 0; i < v; i++)
{
    int vote;
    scanf("%d", &vote);

    if (vote >= 1 && vote <= c)
    {
        count[vote - 1]++;
        valid++;
    }
    else
        invalid++;
}

// Sort candidates by votes
```

```
    for (int i = 0; i < c - 1; i++)
    {
        for (int j = i + 1; j < c; j++)
        {
            if (count[i] < count[j])
            {
                int t = count[i];
                count[i] = count[j];
                count[j] = t;

                t = id[i];
                id[i] = id[j];
                id[j] = t;
            }
        }
    }

    printf("\nValid Votes: %d\n", valid);
    printf("Invalid Votes: %d\n", invalid);

    printf("\nElection Results:\n");

    for (int i = 0; i < c; i++)
        printf("Candidate %d: %d votes\n", id[i], count[i]);

    if (count[0] == count[1] && c > 1)
        printf("Result: Tie\n");
    else
        printf("Winner: Candidate %d\n", id[0]);

    return 0;
}
```

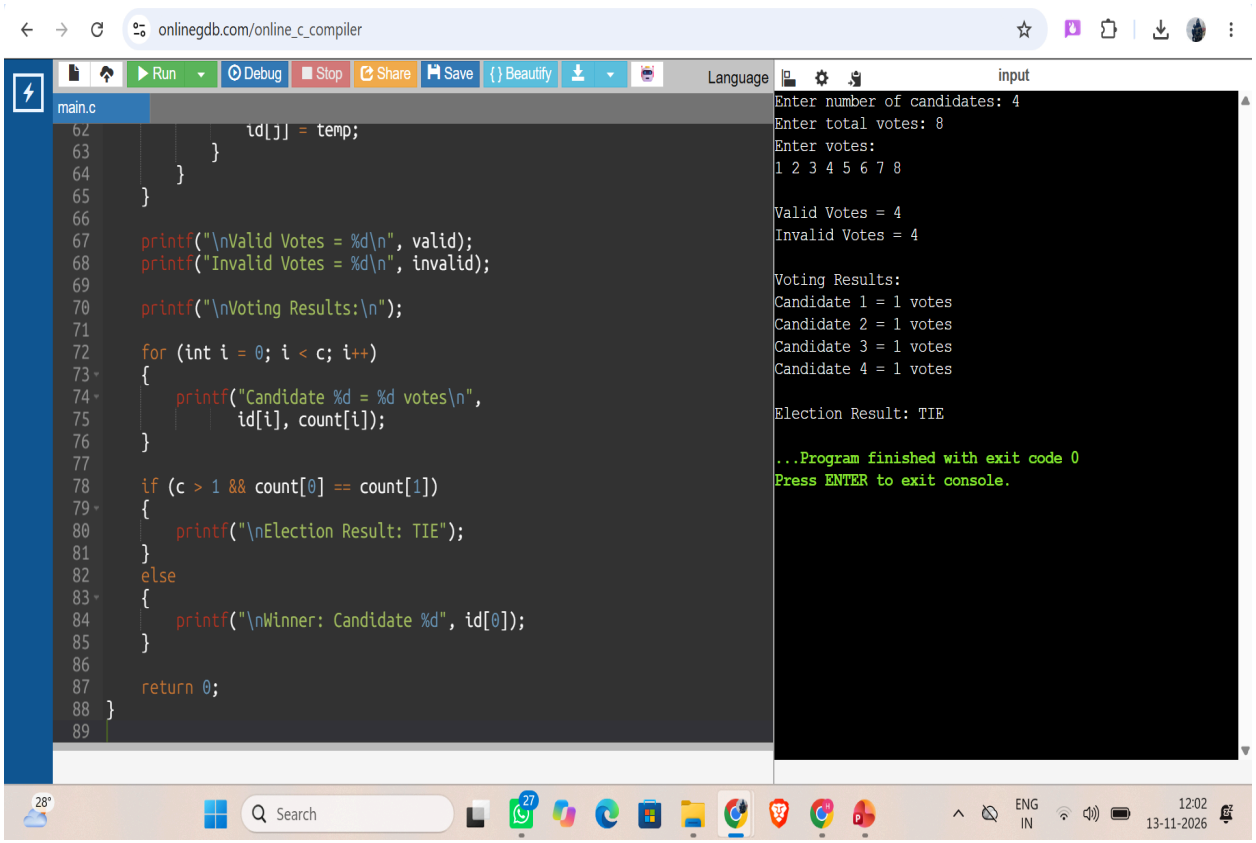
4. Explanation

- `c` represents the number of candidates.
- `v` represents the total number of votes.
- `count[12]` stores the votes received by each candidate.
- `id[12]` stores the candidate numbers.
- `valid` counts the accepted votes.
- `invalid` counts incorrect votes.
- The `if` condition checks whether the entered candidate number is within the valid range.
- The sorting section arranges candidates in descending order of votes.
- Both candidate numbers and vote counts are swapped together.
- The final condition checks whether the highest vote counts are equal.
- If they are equal, the program displays a tie; otherwise, it displays the winner.

5. Output

input:

```
None
Enter number of candidates: 3
Enter total votes: 8
Enter votes:
1 2 1 3 2 1 0 4
```



6. Time Complexity

- Vote counting: O(v)
- Sorting candidates: O(c²)
- Extra space: O(c)

Result: The election counting program was successfully developed to calculate valid votes, invalid votes, candidate totals, winner and tie conditions.